Algorithm Deconstruction Journal – Task Priority Sorting Algorithm

This algorithm uses a clever scoring mechanism to order activities according to significance. To decide which chores you should prioritise first, it considers several factors rather than just the priority level.

Simply said, it's like a teacher choosing which assignments to grade first depending on deadlines, completion status, and perceived urgency in addition to the significance of the subject.

1. **<u>Score Calculation (calculation_task_score)</u>**

   The first step involves assigning a base score determined by the task's priority level:

   - LOW → 10
   - MEDIUM → 20
   - HIGH → 40
   - URGENT→ 60

   This ensures that higher priority tasks start with a higher baseline.

   After setting the base score, the algorithm increases the score depending on close the task is to it's deadline:

   - OVERDUE → +35
   - Due today → +20
   - Due in 1 - 2 days → +15
   - Due in 3 - 7 days→ +10

   This adjustment increases the the urgency of tasks that are approaching their due dates.

   Next, the algorithm applies score reductions based on task completion status:

   - DONE tasks → – 50
   - REVIEW task → –15

   This step lowers the importance of the tasks that are already finished or nearly completed.

Additional increases are applied based on specific conditions:

- Tasks tagged as “bloacker”, “critical” , or “urgent” → +8
- Tasks updated within the last 24 hours → +15

These additions highlight tasks that are important or actively being worked on.

At the end of the process, the function produces a final score that reflects the overall importance of the task.

2. **<u>Sorting Tasks (sort_tasks_by_importance)</u>**

Once the scores have been calculated the algorithm performs this sorting process:

- Scores are generated for every task
- Each task is linked with its calculated score
- Tasks are arranged in descending order, meaning the highest scores appear first.

This guarantees that the most critical tasks are displayed at the top of the list.

3. **<u>Getting Top Tasks (get_top_priority_tasks)</u>**

This stage focuses on selecting only the most important tasks:

- The sorting function is executed
- Only the top N tasks are returned
- The default number of tasks shown in 5

This helps users concentrate on the most urgent tasks without reviewing the entire list.

Visual Representation (Conceptual Flow)

Task → Calculate Score → Add Factors → Final score
↓
All Tasks → Sort by Score (Descending)
↓
Top N Tasks Selected

Key Insights & Learning Points

- The algorithm uses weighted score system which is more flexible than simple sorting.
- It considers several real-world elements such as urgency, importance, and activity.
- The structure allows easy extension (e.g adding new scoring rules)
- The combination of penalties and bonuses results in a more balanced task prioritisation system.

Reflection

How AI Explanation Helped

Using AI explanations made it simpler to understand how different elements combine to produce a single importance score. At first the algorithm appeared complicated but analyzing it step by step made the overall logic easier to follow.

Difficult aspects

Some of the more challenging areas included:

- Understanding how various scoring weights influence the final ranking.
- Imagining how several conditions work together to affect the total score.

How I would explain it to another Junior Developer

I would describe the algorithm in simple terms as follows:

“Every task receives a score depending on how urgent and important it is. Completed tasks lose points, but those with tighter deadlines or higher importance receive more points. To determine which tasks to concentrate on first, we calculate the scores for each task and then arrange them from the highest to lowest.”

Did I test my understanding against AI

Yes, AI tools were used to strengthen my understanding by:

- Breaking down each component of the algorithm
- Confirming the logic behind the scoring calculations
- Reviewing and validating my explanation

Possible Improvements

Several enhancements could improve the efficiency and flexibility of the algorithm:

- Allow scoring weights to be configurable instead of fixed.
- Minimise repeated calls to the current time function to improve performance.
- Improve readability by dividing scoring logic into smaller helper functions.
- Add clearer comments and documentation to support understanding and maintenance